

Simple Linear Regression

Rafiq Islam

2024-08-29

Table of contents

Simple Linear Regression	1
Assumptions of Linear Regressions	4
Synthetic Data	4
Model	5

Simple Linear Regression

A simple linear regression in multiple predictors/input variables/features/independent variables/explanatory variables/regressors/ covariates (many names) often takes the form

$$y = f(\mathbf{x}) + \epsilon = \beta\mathbf{x} + \epsilon$$

where $\beta \in \mathbb{R}^d$ are regression parameters or constant values that we aim to estimate and $\epsilon \sim \mathcal{N}(0, 1)$ is a normally distributed error term independent of x or also called the white noise.

In this case, the model:

$$y = f(x) + \epsilon = \beta_0 + \beta_1 x + \epsilon$$

Therefore, in our model we need to estimate the parameters β_0, β_1 . The true relationship between the explanatory variables and the dependent variable is $y = f(x)$. But our model is $y = f(x) + \epsilon$. Here, this $f(x)$ is the working model with the data. In other words, $\hat{y} = f(x) = \hat{\beta}_0 + \hat{\beta}_1 x$. Therefore, there should be some error in the model prediction which we are calling $\epsilon = \|y - \hat{y}\|$ where y is the true value and $\hat{y}$ is the predicted value. This error term is normally distributed with mean 0 and variance 1. To get the best estimate of the parameters

β_0, β_1 we can minimize the error term as much as possible. So, we define the residual sum of squares (RSS) as:

$$RSS = \epsilon_1^2 + \epsilon_2^2 + \cdots + \epsilon_{10}^2 \quad (1)$$

$$= \sum_{i=1}^{10} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2 \quad (2)$$

$$\hat{\Downarrow}(\bar{\beta}) = \sum_{i=1}^{10} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2 \quad (3)$$

$$(4)$$

Using multivariate calculus we see

$$\frac{\partial l}{\partial \beta_0} = \sum_{i=1}^{10} 2(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)(-1) \quad (5)$$

$$\frac{\partial l}{\partial \beta_1} = \sum_{i=1}^{10} 2(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)(-x_i) \quad (6)$$

Setting the partial derivatives to zero we solve for $\hat{\beta}_0, \hat{\beta}_1$ as follows

$$\begin{aligned} \frac{\partial l}{\partial \beta_0} &= 0 \\ \Rightarrow \sum_{i=1}^{10} y_i - 10\hat{\beta}_0 - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i \right) &= 0 \\ \Rightarrow \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x} \end{aligned}$$

and,

$$\begin{aligned}
& \frac{\partial l}{\partial \beta_1} = 0 \\
& \Rightarrow \sum_{i=1}^{10} 2(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)(-x_i) = 0 \\
& \Rightarrow \sum_{i=1}^{10} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)(x_i) = 0 \\
& \Rightarrow \sum_{i=1}^{10} x_i y_i - \hat{\beta}_0 \left(\sum_{i=1}^{10} x_i \right) - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i^2 \right) = 0 \\
& \Rightarrow \sum_{i=1}^{10} x_i y_i - (\bar{y} - \hat{\beta}_1 \bar{x}) \left(\sum_{i=1}^{10} x_i \right) - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i^2 \right) = 0 \\
& \Rightarrow \sum_{i=1}^{10} x_i y_i - \bar{y} \left(\sum_{i=1}^{10} x_i \right) + \hat{\beta}_1 \bar{x} \left(\sum_{i=1}^{10} x_i \right) - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i^2 \right) = 0 \\
& \Rightarrow \sum_{i=1}^{10} x_i y_i - \bar{y} \left(\sum_{i=1}^{10} x_i \right) - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i^2 - \bar{x} \sum_{i=1}^{10} x_i \right) = 0 \\
& \Rightarrow \sum_{i=1}^{10} x_i y_i - \bar{y} \left(\sum_{i=1}^{10} x_i \right) - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i^2 - 10\bar{x}^2 \right) = 0 \\
& \Rightarrow \sum_{i=1}^{10} x_i y_i - \bar{y} \left(\sum_{i=1}^{10} x_i \right) - \hat{\beta}_1 \left(\sum_{i=1}^{10} x_i^2 - 2 \times 10 \times \bar{x}^2 + 10\bar{x}^2 \right) = 0 \\
& \Rightarrow \hat{\beta}_1 = \frac{\sum_{i=1}^{10} x_i y_i - 10\bar{x}\bar{y}}{\sum_{i=1}^{10} x_i^2 - 2 \times 10 \times \bar{x}^2 + 10\bar{x}^2} \\
& \Rightarrow \hat{\beta}_1 = \frac{\sum_{i=1}^{10} x_i y_i - 10\bar{x}\bar{y} - 10\bar{x}\bar{y} + 10\bar{x}\bar{y}}{\sum_{i=1}^{10} x_i^2 - 2\bar{x} \times 10 \times \frac{1}{10} \sum_{i=1}^{10} x_i + 10\bar{x}^2} \\
& \Rightarrow \hat{\beta}_1 = \frac{\sum_{i=1}^{10} x_i y_i - \bar{y} \left(\sum_{i=1}^{10} x_i \right) - \bar{x} \left(\sum_{i=1}^{10} y_i \right) + 10\bar{x}\bar{y}}{\sum_{i=1}^{10} (x_i - \bar{x})^2} \\
& \Rightarrow \hat{\beta}_1 = \frac{\sum_{i=1}^{10} (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y})}{\sum_{i=1}^{10} (x_i - \bar{x})^2} \\
& \Rightarrow \hat{\beta}_1 = \frac{\sum_{i=1}^{10} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{10} (x_i - \bar{x})^2}
\end{aligned}$$

Therefore, we have the following

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{10} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{10} (x_i - \bar{x})^2}$$

Simple Linear Regression `slr` is applicable for a single feature data set with continuous response variable.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

Assumptions of Linear Regressions

- **Linearity:** The relationship between the feature set and the target variable has to be linear.
- **Homoscedasticity:** The variance of the residuals has to be constant.
- **Independence:** All the observations are independent of each other.
- **Normality:** The distribution of the dependent variable y has to be normal.

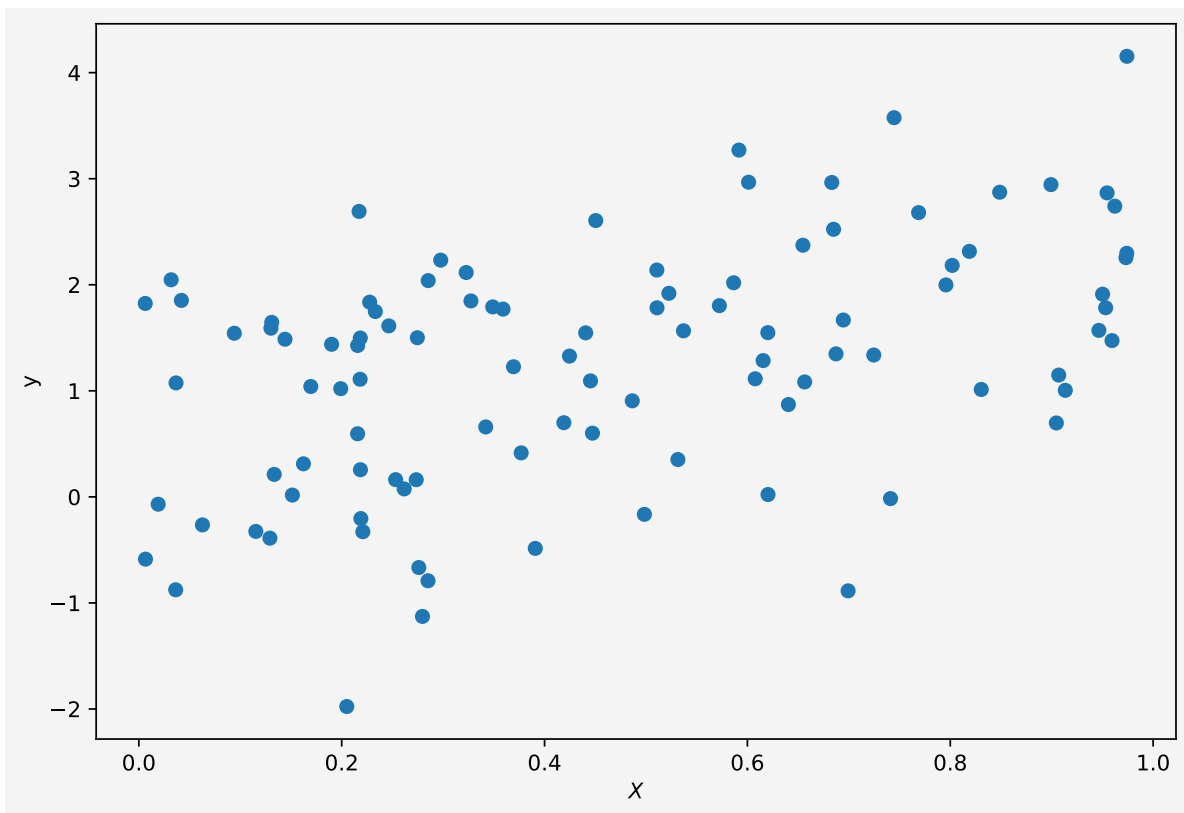
Synthetic Data

To implement the algorithm, we need some synthetic data. To generate the synthetic data we use the linear equation $y(x) = 2x + \frac{1}{2} + \xi$ where $\xi \sim \mathbf{N}(0, 1)$

```
X=np.random.random(100)
y=2*X+0.5+np.random.randn(100)
```

Note that we used two random number generators, `np.random.random(n)` and `np.random.randn(n)`. The first one generates n random numbers of values from the range (0,1) and the second one generates values from the standard normal distribution with mean 0 and variance or standard deviation 1.

```
plt.figure(figsize=(9,6))
plt.scatter(X,y)
plt.xlabel('$X$')
plt.ylabel('y')
plt.gca().set_facecolor('#f4f4f4')
plt.gcf().patch.set_facecolor('#f4f4f4')
plt.show()
```



Model

We want to fit a simple linear regression to the above data.

```
slr=LinearRegression()
```

Now to fit our data X and y we need to reshape the input variable. Because if we look at X ,

X

```
array([0.13100955, 0.44055146, 0.95946128, 0.25318279, 0.19907474,
       0.97402426, 0.42456165, 0.32741532, 0.768776 , 0.03184284,
       0.64040627, 0.81884422, 0.97318853, 0.0419508 , 0.95460592,
       0.21712834, 0.61550075, 0.34888841, 0.51086491, 0.69455316,
       0.21573333, 0.65644046, 0.27348575, 0.00630906, 0.90459473,
       0.84870598, 0.27604573, 0.44528659, 0.22760601, 0.51070629,
       0.53699371, 0.36938635, 0.34199772, 0.28509116, 0.39085432,
       0.52256523, 0.57236023, 0.60116741, 0.21846489, 0.06269154,
       0.01904812, 0.95319529, 0.49844307, 0.95017294, 0.11533462,
       0.97413924, 0.23314875, 0.32272736, 0.29763125, 0.94652544,
       0.03666339, 0.59163019, 0.22082909, 0.62029682, 0.26162661,
       0.28528115, 0.48647136, 0.20500826, 0.91341167, 0.72457624,
       0.44724081, 0.15136118, 0.74107663, 0.74459206, 0.21568678,
       0.60768711, 0.19005142, 0.96222616, 0.68494314, 0.68734413,
       0.65471488, 0.6831679 , 0.12923082, 0.58641442, 0.41901423,
       0.14410887, 0.83056738, 0.89926586, 0.03633761, 0.09410449,
       0.90690883, 0.37695309, 0.5314666 , 0.62013194, 0.27459944,
       0.450477 , 0.24642473, 0.13023584, 0.0065608 , 0.80190561,
       0.16226238, 0.21816952, 0.69925209, 0.21890176, 0.35905451,
       0.13345548, 0.79573236, 0.1695548 , 0.27963723, 0.218332  ])
```

It is a one-dimensional array/vector but the `slr` object accepts input variable as matrix or two-dimensional format.

```
X=X.reshape(-1,1)
X[:10]
```

```
array([[0.13100955],
       [0.44055146],
       [0.95946128],
       [0.25318279],
       [0.19907474],
       [0.97402426],
       [0.42456165],
       [0.32741532],
       [0.768776  ],
       [0.03184284]])
```

Now we fit the data to our model

```
slr.fit(X,y)
slr.predict([[2],[3]])
```

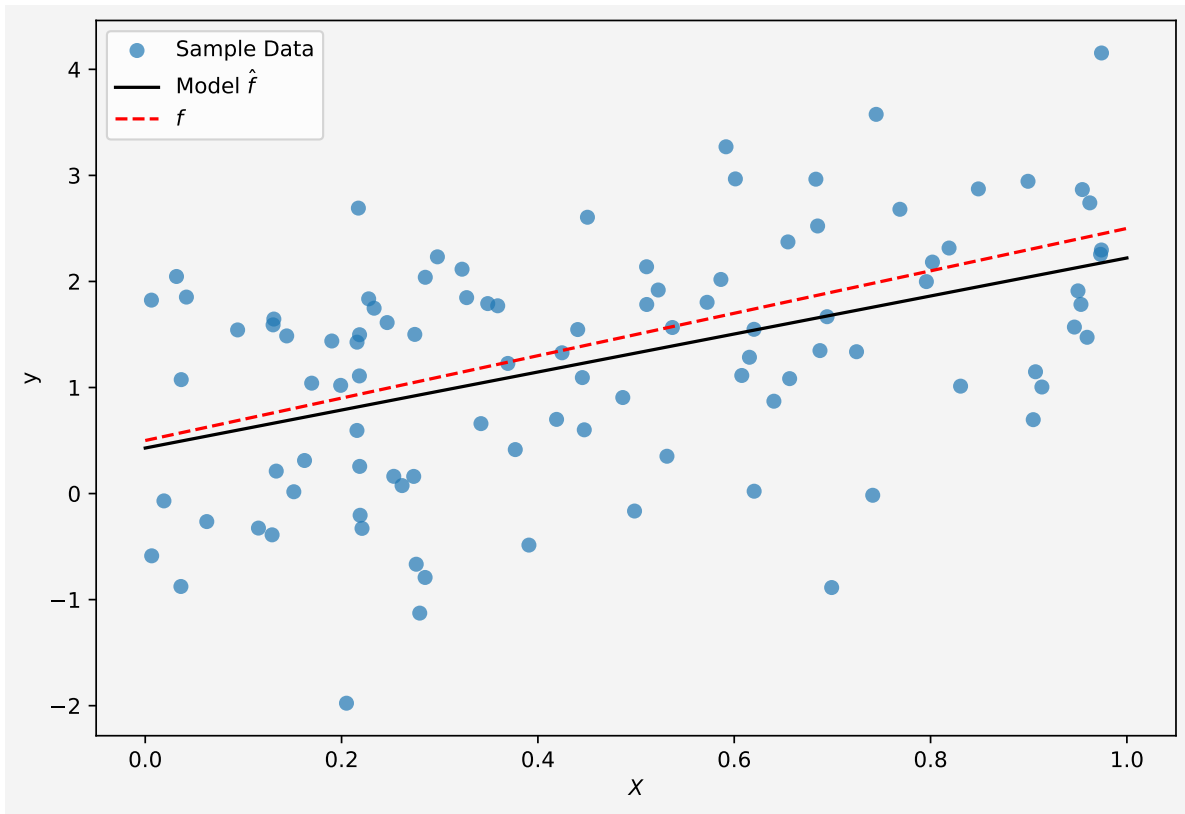
```
array([4.01348089, 5.80557844])
```

We have our $X = 2, 3$ and the corresponding y values are from the above cell output, which are pretty close to the model $y = 2x + \frac{1}{2}$.

```
intercept = round(slr.intercept_,4)
slope = slr.coef_
```

Now our model parameters are: intercept $\beta_0 = \text{np.float64}(0.4293)$ and slope $\beta_1 = \text{array}([1.79209755])$.

```
plt.figure(figsize=(9,6))
plt.scatter(X,y, alpha=0.7,label="Sample Data")
plt.plot(np.linspace(0,1,100),
         slr.predict(np.linspace(0,1,100).reshape(-1,1)),
         'k',
         label='Model  $\hat{f}$ ')
)
plt.plot(np.linspace(0,1,100),
         2*np.linspace(0,1,100)+0.5,
         'r--',
         label='$f$')
)
plt.xlabel('$X$')
plt.ylabel('$y$')
plt.legend(fontsize=10)
plt.gca().set_facecolor('#f4f4f4')
plt.gcf().patch.set_facecolor('#f4f4f4')
plt.show()
```



So the model fits the data almost perfectly.

Up next [multiple linear regression](#).

Share on



You may also like